多租户核心基础组件实现代码

一、后端可落地实现代码（完全兼容现有Spring Boot + MyBatis-Plus结构）

以下代码均按照你现有「实体-Mapper-Service直接实现-Controller」的分层结构编写，无需调整架构即可接入。

1. 多租户核心基础组件

1.1 租户上下文工具（`util/TenantContext.java`）

基于ThreadLocal存储当前请求的租户ID，请求进出时设置/清除，业务代码无感知。

代码块

```java
package com.yourproject.util;

public class TenantContext {
    private static final ThreadLocal<Long> TENANT_HOLDER = new ThreadLocal<>();

    public static void setTenantId(Long tenantId) {
        TENANT_HOLDER.set(tenantId);
    }

    public static Long getTenantId() {
        return TENANT_HOLDER.get();
    }

    public static void clear() {
        TENANT_HOLDER.remove();
    }
}
```

1.2 租户拦截器（`interceptor/TenantInterceptor.java`）

从请求头JWT中解析租户ID，校验租户状态，写入上下文，实现全链路租户透传。

代码块

```java
package com.yourproject.interceptor;

import cn.hutool.core.util.StrUtil;
```

```java
import com.yourproject.entity.Tenant;
import com.yourproject.service.TenantService;
import com.yourproject.util.JwtUtil;
import com.yourproject.util.TenantContext;
import org.slf4j.MDC;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@Component
public class TenantInterceptor implements HandlerInterceptor {

    @Autowired
    private TenantService tenantService;

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) throws Exception {
        String token = request.getHeader("Authorization");
        if (StrUtil.isBlank(token)) {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.setContentType("application/json;charset=UTF-8");
            response.getWriter().write("{\"code\":401,\"msg\":\"未登录，请先登录\"}");
            return false;
        }

        Long tenantId = JwtUtil.getTenantId(token);
        Long userId = JwtUtil.getUserId(token);
        if (tenantId == null || userId == null) {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.setContentType("application/json;charset=UTF-8");
            response.getWriter().write("{\"code\":401,\"msg\":\"登录状态无效，请重新登录\"}");
            return false;
        }

        // 平台管理员（tenantId=0）跳过租户状态校验
        if (0L != tenantId) {
            Tenant tenant = tenantService.getById(tenantId);
            if (tenant == null || tenant.getStatus() != 1) {
                response.setStatus(HttpServletResponse.SC_FORBIDDEN);
                response.setContentType("application/json;charset=UTF-8");
```

```
                response.getWriter().write("{\"code\":403,\"msg\":\"租户已停用或不存在，请联系平台管理员\"}");
                return false;
            }
        }

        TenantContext.setTenantId(tenantId);
        MDC.put("tenantId", String.valueOf(tenantId)); // 日志埋点
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse response, Object handler, Exception ex) {
        TenantContext.clear();
        MDC.remove("tenantId");
    }
}
```

1.3 MVC拦截器注册（`config/WebMvcConfig.java`）

代码块

```
package com.yourproject.config;

import com.yourproject.interceptor.TenantInterceptor;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    @Autowired
    private TenantInterceptor tenantInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(tenantInterceptor)
                .addPathPatterns("/api/**")
                .excludePathPatterns(
                        "/api/auth/login",
                        "/api/auth/captcha",
                        "/static/**",
                        "/error"
```

```
                );
        }
    }
```

1.4 MyBatis-Plus多租户+分页一体化配置（`config/MyBatisPlusConfig.java`）

整合原有分页配置，加入多租户拦截器，自动拼接 `tenant_id` 条件，业务代码零侵入。

代码块

```
package com.yourproject.config;

import com.baomidou.mybatisplus.annotation.DbType;
import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.TenantLineInnerInterceptor;
import com.yourproject.util.TenantContext;
import net.sf.jsqlparser.expression.LongValue;
import net.sf.jsqlparser.expression.NullValue;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.Arrays;
import java.util.List;

@Configuration
public class MyBatisPlusConfig {

    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();

        // 多租户拦截器（必须放在分页拦截器之前）
        TenantLineInnerInterceptor tenantInterceptor = new TenantLineInnerInterceptor();
        tenantInterceptor.setTenantLineHandler(new com.baomidou.mybatisplus.extension.plugins.handler.TenantLineHandler() {
            @Override
            public net.sf.jsqlparser.expression.Expression getTenantId() {
                Long tenantId = TenantContext.getTenantId();
                // 平台管理员返回null，不拼接租户条件，查询全局数据
                return tenantId == null || tenantId == 0 ? new NullValue() : new LongValue(tenantId);
            }
```

```
33
34              @Override
35              public String getTenantIdColumn() {
36                  return "tenant_id";
37              }
38
39              @Override
40              public boolean ignoreTable(String tableName) {
41                  // 全局平台表不参与租户隔离
42                  List<String> ignoreTables = Arrays.asList(
43                          "sys_tenant",
44                          "sys_package",
45                          "sys_platform_admin",
46                          "sys_dict"
47                  );
48                  return ignoreTables.contains(tableName);
49              }
50          });
51          interceptor.addInnerInterceptor(tenantInterceptor);
52
53          // 分页拦截器
54          interceptor.addInnerInterceptor(new
    PaginationInnerInterceptor(DbType.MYSQL));
55
56          return interceptor;
57      }
58  }
```

2. 租户管理基础代码

2.1 租户实体类（`entity/Tenant.java`）

对应数据库表 `sys_tenant`

代码块

```
1   package com.yourproject.entity;
2
3   import com.baomidou.mybatisplus.annotation.IdType;
4   import com.baomidou.mybatisplus.annotation.TableId;
5   import com.baomidou.mybatisplus.annotation.TableName;
6   import lombok.Data;
7
8   import java.time.LocalDateTime;
9
10  @Data
11  @TableName("sys_tenant")
```

```
public class Tenant {
    @TableId(type = IdType.AUTO)
    private Long id;
    private String tenantCode;
    private String orgName;
    private String contact;
    private String phone;
    private Integer status;
    private Long packageId;
    private LocalDateTime expireTime;
    private LocalDateTime createTime;
    private LocalDateTime updateTime;
}
```

2.2 租户Mapper（`mapper/TenantMapper.java`）

代码块

```
package com.yourproject.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yourproject.entity.Tenant;
import org.apache.ibatis.annotations.Mapper;

@Mapper
public interface TenantMapper extends BaseMapper<Tenant> {
}
```

2.3 租户Service（`service/TenantService.java`）

采用直接实现模式，不定义接口。

代码块

```
package com.yourproject.service;

import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.yourproject.entity.Tenant;
import com.yourproject.mapper.TenantMapper;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class TenantService {

    @Autowired
```

```
    private TenantMapper tenantMapper;

    public Tenant getById(Long id) {
        return tenantMapper.selectById(id);
    }

    public Tenant getByCode(String tenantCode) {
        LambdaQueryWrapper<Tenant> wrapper = new LambdaQueryWrapper<>();
        wrapper.eq(Tenant::getTenantCode, tenantCode);
        return tenantMapper.selectOne(wrapper);
    }
}
```

3. 认证体系适配

3.1 JWT工具类（`util/JwtUtil.java`）

基于Hutool JWT实现，生成携带租户ID、用户ID、角色的令牌。

代码块

```
package com.yourproject.util;

import cn.hutool.jwt.JWT;
import cn.hutool.jwt.JWTUtil;
import cn.hutool.jwt.signers.JWTSigner;
import cn.hutool.jwt.signers.JWTSignerUtil;

import java.nio.charset.StandardCharsets;
import java.util.Date;

public class JwtUtil {
    private static final String SECRET = "your-jwt-secret-key-2024";
    private static final JWTSigner = JWTSignerUtil.hs256(SECRET.getBytes(StandardCharsets.UTF_8));
    private static final long EXPIRE_TIME = 24 * 60 * 60 * 1000L;

    public static String createToken(Long tenantId, Long userId, String role, String userName) {
        return JWT.create()
                .setPayload("tenantId", tenantId)
                .setPayload("userId", userId)
                .setPayload("role", role)
                .setPayload("userName", userName)
                .setExpiresAt(new Date(System.currentTimeMillis() + EXPIRE_TIME))
                .sign(SIGNER);
```

```
    }

    public static Long getTenantId(String token) {
        try {
            JWT jwt = JWTUtil.parseToken(token);
            Object val = jwt.getPayload("tenantId");
            return val == null ? null : Long.valueOf(val.toString());
        } catch (Exception e) {
            return null;
        }
    }

    public static Long getUserId(String token) {
        try {
            JWT jwt = JWTUtil.parseToken(token);
            Object val = jwt.getPayload("userId");
            return val == null ? null : Long.valueOf(val.toString());
        } catch (Exception e) {
            return null;
        }
    }

    public static String getRole(String token) {
        try {
            JWT jwt = JWTUtil.parseToken(token);
            Object val = jwt.getPayload("role");
            return val == null ? null : val.toString();
        } catch (Exception e) {
            return null;
        }
    }

    public static boolean verify(String token) {
        try {
            return JWTUtil.verify(token, SIGNER);
        } catch (Exception e) {
            return false;
        }
    }
}
```

3.2 登录接口改造（`controller/AuthController.java`）

增加租户编码参数，支持多角色统一登录入口。

```
代码块
```

```java
package com.yourproject.controller;

import cn.hutool.core.util.StrUtil;
import com.yourproject.entity.Tenant;
import com.yourproject.service.TenantService;
import com.yourproject.util.JwtUtil;
import com.yourproject.util.Result;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private TenantService tenantService;

    @PostMapping("/login")
    public Result<Map<String, Object>> login(
            @RequestParam String tenantCode,
            @RequestParam String username,
            @RequestParam String password,
            @RequestParam String role) {

        Long tenantId;
        String userName;

        // 平台管理员登录
        if ("platform".equals(tenantCode) && "platform_admin".equals(role)) {
            if (!"admin".equals(username) || !"admin123".equals(password)) {
                return Result.error("账号或密码错误");
            }
            tenantId = 0L;
            userName = "平台管理员";
        } else {
            // 租户端登录：先校验租户状态
            Tenant tenant = tenantService.getByCode(tenantCode);
            if (tenant == null || tenant.getStatus() != 1) {
                return Result.error("租户编码无效或已停用");
            }
            tenantId = tenant.getId();

            // 根据角色校验对应账号（教师/学生/租户管理员，补充tenant_id条件查询）
            if ("teacher".equals(role)) {
```

```
                // 教师账号校验逻辑
                userName = "教师用户";
            } else if ("student".equals(role)) {
                // 学生账号校验逻辑
                userName = "学生用户";
            } else if ("admin".equals(role)) {
                // 租户管理员校验逻辑
                userName = "租户管理员";
            } else {
                return Result.error("角色类型错误");
            }
        }

        String token = JwtUtil.createToken(tenantId, 1L, role, userName);

        Map<String, Object> data = new HashMap<>();
        data.put("token", token);
        data.put("role", role);
        data.put("userName", userName);
        data.put("tenantCode", tenantCode);

        return Result.success(data);
    }
}
```

4. 业务层改造示例（以CourseService为例）

保留原有直接实现结构，新增套餐权益校验逻辑。

代码块

```
@Service
public class CourseService {

    @Autowired
    private CourseMapper courseMapper;

    @Autowired
    private TenantPackageService tenantPackageService;

    public PageResult<CourseVO> getCoursePage(CourseQuery query) {
        Page<Course> page = new Page<>(query.getPageNum(),
query.getPageSize());
        LambdaQueryWrapper<Course> wrapper = new LambdaQueryWrapper<>();

        if (StrUtil.isNotBlank(query.getCourseName())) {
            wrapper.like(Course::getCourseName, query.getCourseName());
```

```
        }
        wrapper.orderByDesc(Course::getCreateTime);

        Page<Course> resultPage = courseMapper.selectPage(page, wrapper);
        List<CourseVO> voList = BeanUtil.copyToList(resultPage.getRecords(),
CourseVO.class);
        return PageResult.of(resultPage).setRows(voList);
    }

    public boolean save(Course course) {
        // 校验当前租户课程数量是否超出套餐上限
        int currentCount = courseMapper.selectCount(null);
        int maxCount =
tenantPackageService.getMaxCourseCount(TenantContext.getTenantId());

        if (currentCount >= maxCount) {
            throw new BusinessException("课程数量已达套餐上限，请升级套餐后再创建");
        }

        return courseMapper.insert(course) > 0;
    }
}
```

二、前端可落地实现代码（贴合现有原生JS动态页面结构）

1. 登录页改造（`index.html`）

在原有表单中增加租户编码输入项。

代码块

```
<div class="form-item">
    <label><i class="fa fa-building"></i> 租户编码</label>
    <input type="text" id="tenant-code" placeholder="请输入租户编码">
    <div class="error-tip" id="tenant-error"></div>
</div>
```

登录提交逻辑同步修改：

代码块

```
async function handleLogin() {
    const tenantCode = document.getElementById('tenant-code').value.trim();
    const username = document.getElementById('username').value.trim();
```

```
    const password = document.getElementById('password').value;
    const role = document.getElementById('user-role').value;

    if (!tenantCode) {
        showError('tenant-error', '请输入租户编码');
        return;
    }

    const formData = new URLSearchParams();
    formData.append('tenantCode', tenantCode);
    formData.append('username', username);
    formData.append('password', password);
    formData.append('role', role);

    try {
        const res = await fetch('/api/auth/login', {
            method: 'POST',
            body: formData
        });
        const result = await res.json();

        if (result.code === 200) {
            localStorage.setItem('token', result.data.token);
            localStorage.setItem('userRole', result.data.role);
            localStorage.setItem('tenantCode', result.data.tenantCode);
            localStorage.setItem('userName', result.data.userName);

            redirectByRole(result.data.role);
        } else {
            showError('login-error', result.msg);
        }
    } catch (e) {
        showError('login-error', '登录请求失败，请稍后重试');
    }
}
```

2. 全局请求封装（公共JS文件）

统一封装接口请求，自动携带认证头，统一处理401跳转。

代码块

```
const http = {
    baseUrl: '',

    async get(url, params = {}) {
        const query = new URLSearchParams(params).toString();
```

```
        const fullUrl = query ? `${this.baseUrl}${url}?${query}` : `${this.baseUrl}${url}`;

        const res = await fetch(fullUrl, {
            headers: {
                'Authorization': localStorage.getItem('token') || '',
                'Content-Type': 'application/json'
            }
        });
        return this.handleResponse(res);
    },

    async post(url, data = {}) {
        const res = await fetch(`${this.baseUrl}${url}`, {
            method: 'POST',
            headers: {
                'Authorization': localStorage.getItem('token') || '',
                'Content-Type': 'application/x-www-form-urlencoded'
            },
            body: new URLSearchParams(data)
        });
        return this.handleResponse(res);
    },

    async handleResponse(res) {
        if (res.status === 401) {
            localStorage.clear();
            window.location.href = '/index.html';
            throw new Error('登录已过期，请重新登录');
        }
        const data = await res.json();
        if (data.code !== 200) {
            throw new Error(data.msg || '请求失败');
        }
        return data;
    }
};
```

使用示例：

代码块

```
// 替换原有fetch写法
const result = await http.get('/api/course/page', {
    pageNum: coursePagination.pageNum,
    pageSize: coursePagination.pageSize,
```

```
    courseName: document.getElementById('course-name-input').value.trim()
});
```

3. 管理端菜单权限适配

根据登录角色动态渲染侧边菜单，区分平台管理员和租户管理员。

代码块

```
function loadSideMenu() {
    const role = localStorage.getItem('userRole');
    const menuList = document.getElementById('menu-list');

    let menus = [];

    if (role === 'platform_admin') {
        menus = [
            { icon: 'fa-tachometer-alt', title: '数据概览', key: 'dashboard' },
            { icon: 'fa-building', title: '租户管理', key: 'tenant' },
            { icon: 'fa-th-large', title: '套餐管理', key: 'package' },
            { icon: 'fa-chart-line', title: '运营统计', key: 'statistics' },
            { icon: 'fa-cog', title: '系统设置', key: 'settings' }
        ];
    } else if (role === 'admin') {
        menus = [
            { icon: 'fa-tachometer-alt', title: '首页概览', key: 'dashboard' },
            { icon: 'fa-book-open', title: '课程管理', key: 'course' },
            { icon: 'fa-chalkboard-teacher', title: '教师管理', key: 'teacher' },
            { icon: 'fa-user-graduate', title: '学生管理', key: 'student' },
            { icon: 'fa-calendar-check', title: '预约管理', key: 'reservation' },
            { icon: 'fa-chart-bar', title: '数据统计', key: 'report' }
        ];
    }

    menuList.innerHTML = menus.map(item => `
        <li class="menu-item" data-key="${item.key}" onclick="switchMenu('${item.key}')">
            <i class="fa ${item.icon}"></i>
            <span>${item.title}</span>
        </li>
    `).join('');
}
```

三、运维监控可落地实现代码

1. 应用监控配置（`application.yml`）

启用Spring Boot Actuator，独立监控端口。

代码块

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,logfile
  endpoint:
    health:
      show-details: always
  server:
    port: 8081
```

可访问监控项：

- 健康检查：`http://localhost:8081/actuator/health`
- 应用指标：`http://localhost:8081/actuator/metrics`
- 实时日志：`http://localhost:8081/actuator/logfile`

2. 带租户标识的日志配置（`logback-spring.xml`）

日志格式内置租户ID，便于按租户排查问题。

代码块

```xml
<configuration>
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] [%X{tenantId}] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <root level="info">
        <appender-ref ref="CONSOLE"/>
    </root>
</configuration>
```

3. 租户级轻量限流（基于Guava）

按租户维度限制请求频率，避免单租户占用全部资源。

代码块

```
@Component
public class RateLimitInterceptor implements HandlerInterceptor {
    private final com.google.common.cache.LoadingCache<Long,
com.google.common.util.concurrent.RateLimiter> rateLimiterCache =
            com.google.common.cache.CacheBuilder.newBuilder()
                    .expireAfterAccess(10,
java.util.concurrent.TimeUnit.MINUTES)
                    .build(new com.google.common.cache.CacheLoader<Long,
com.google.common.util.concurrent.RateLimiter>() {
                        @Override
                        public com.google.common.util.concurrent.RateLimiter
load(Long tenantId) {
                            return
com.google.common.util.concurrent.RateLimiter.create(100); // 每秒100请求
                        }
                    });

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler) throws Exception {
        Long tenantId = TenantContext.getTenantId();
        if (tenantId == null || tenantId == 0) return true;

        com.google.common.util.concurrent.RateLimiter limiter =
rateLimiterCache.get(tenantId);
        if (!limiter.tryAcquire()) {
            response.setStatus(429);
            response.setContentType("application/json;charset=UTF-8");
            response.getWriter().write("{\"code\":429,\"msg\":\"请求过于频繁，请
稍后再试\"}");
            return false;
        }
        return true;
    }
}
```

还需要你提供的数据

为了让代码完全适配你的项目并可直接编译运行，还需要补充以下信息：

项目基础信息

- 项目完整包名（如 `com.language.reservation`）
- Spring Boot、MyBatis-Plus、Hutool的具体版本号
- 现有统一返回结果类 `Result` 的完整包路径和字段结构

数据库相关

- 教师表、学生表、管理员表的完整字段结构
- 所有业务表是否已添加 `tenant_id` 字段及索引
- 平台管理员表、套餐表是否需要从零设计，还是已有初步结构

前端相关

- 现有前端页面的文件目录结构（各HTML、JS文件的存放位置）
- 公共样式、公共JS的文件名和引入方式
- 现有管理端动态内容渲染的入口函数名

业务与部署

- 套餐的具体权益项（课程数量上限、教师数量上限、存储容量等）
- 项目部署方式（jar包运行、Docker、Tomcat）
- 是否需要支持子域名租户识别（如 `xxx.xxx.com` 对应租户）
- 现有短信、文件存储等第三方服务的接入情况

（注：部分内容可能由 AI 生成）